Design, Implementation, and Analysis of a Lightweight Transformer Language Model

Berkay Baris Turan (bt2686)
COMS W4995 031: Applied Machine Learning
Columbia University
Notebook: https://github.com/bturann/transformer-from-scratch-nlp

I. Introduction

Recent advances in natural language processing have been largely driven by the Transformer architecture, which enables efficient modeling of long-range dependencies in sequential data through self-attention mechanisms. Unlike recurrent models, Transformers process tokens in parallel and dynamically learn contextual relationships, making them the foundation of modern large language models.

In this work, a lightweight Transformer-based language model is implemented from scratch and trained on the Tiny Shakespeare dataset. The task is formulated as next-token prediction, also known as autoregressive language modeling, where the model learns to predict the next token given preceding context. This formulation captures fundamental aspects of language, including syntax, structure, and contextual dependencies.

A minimal yet complete Transformer pipeline is constructed, including subword tokenization using Byte Pair Encoding (BPE), causal self-attention, positional encoding, normalization layers, and feed-forward networks. The model is trained using cross-entropy loss and evaluated using perplexity, a standard metric for language modeling performance.

Beyond implementation, the behavior of the model is systematically analyzed. Training dynamics are examined through loss and perplexity curves, while the effects of key hyperparameters such as learning rate, context length, and model size are evaluated. Additionally, attention mechanisms are interpreted through visualization to better understand how the model captures relationships within text.

These results demonstrate that even a compact Transformer model is capable of learning meaningful linguistic patterns and provide insight into the fundamental principles underlying modern language models.

II. Data Preparation

A. Dataset

The model is trained on the Tiny Shakespeare dataset, a commonly used benchmark for character- and subword-level language modeling. The dataset consists of a collection of Shakespearean plays, providing rich linguistic structure, diverse vocabulary, and recurring stylistic patterns. Its relatively small size makes it suitable for lightweight experiments while still preserving meaningful language dependencies.

B. Subword Tokenization

To efficiently represent text, Byte Pair Encoding (BPE) is employed as the tokenization strategy. Instead of operating at the character level, BPE constructs a subword vocabulary by iteratively merging the most frequent pairs of characters or character sequences. This approach enables the model to capture meaningful linguistic units such as common prefixes, suffixes, and frequently occurring word fragments.

The vocabulary size is constrained to 500 tokens to maintain computational efficiency while allowing sufficient expressiveness. Each input sequence is therefore represented as a sequence of integer token indices corresponding to the learned subword vocabulary.

C. Sequence Construction

The tokenized text is divided into fixed-length sequences for training. Each sequence consists of a contiguous block of tokens, and the training objective is formulated as next-token prediction. Specifically, for a given input sequence $(x_1, x_2, \ldots, x_T)$, the target sequence is defined as $(x_2, x_3, \ldots, x_{T+1})$.

To maximize data utilization, overlapping sequences are generated using a sliding window approach. This ensures that each token contributes to multiple training examples, improving the efficiency of learning and stabilizing training dynamics.

D. Train-Validation Split

The dataset is partitioned into training and validation sets using an 80/20 split. The training set is used to optimize model parameters, while the validation set is used to monitor generalization performance and compute evaluation metrics such as validation loss and perplexity.

No separate test set is used, in accordance with the assignment requirements, as the primary focus is on model implementation and analysis rather than final benchmark evaluation.

III. Tiny Transformer Architecture

A. Input Embeddings and Positional Encoding

Each input token is first mapped to a dense vector representation through a learned embedding matrix. These embeddings transform discrete token indices into continuous vector spaces where semantic relationships can be captured.

Since the Transformer architecture does not inherently encode token order, positional information is incorporated using sinusoidal positional encoding. This encoding assigns a unique vector to each position in the sequence, allowing the model to distinguish between tokens based on their relative and absolute positions. The positional encodings are added to the token embeddings to produce the final input representations.

Positional encoding is essential for capturing sequence order. Without it, the model would treat tokens as an unordered set, making it unable to distinguish between different word arrangements. Although training would still be possible, performance would degrade significantly, as meaningful sequential relationships could not be learned.

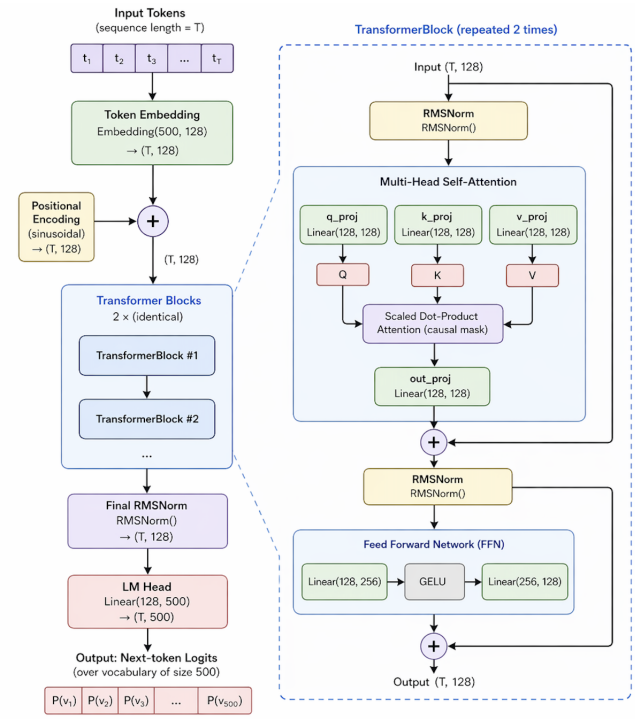

Fig. 1: Architecture of the implemented Tiny Transformer model.

B. Causal Self-Attention Mechanism

The core component of the model is the causal self-attention mechanism, which enables each token to attend to previous tokens in the sequence. For a given sequence, queries, keys, and values are computed as linear projections of the input embeddings.

Attention scores are obtained by computing scaled dot-products between queries and keys. A causal mask is applied to prevent each position from attending to future tokens, ensuring that the model adheres to the autoregressive nature of the task. The masked scores are then normalized using a softmax function, producing attention weights that determine how much information is aggregated from each token.

The output of the attention mechanism is computed as a weighted sum of the value vectors. Multi-head attention is employed to allow the model to capture different types of relationships in parallel, with each head learning distinct attention patterns.

C. Feed-Forward Network and Normalization

Following the attention mechanism, each token representation is passed through a position-wise feed-forward network consisting of two linear transformations with a non-linear activation in between. This component enhances the model's capacity to learn complex transformations of the input features.

To stabilize training and improve convergence, RMSNorm is applied to the inputs of each sublayer. Residual connections are also incorporated, allowing the model to preserve information from earlier layers while learning incremental refinements. These design choices help mitigate issues such as vanishing gradients and enable deeper architectures.

D. Model Configuration

The final model consists of two stacked Transformer blocks, each containing a causal self-attention layer and a feed-forward network. The architecture is intentionally kept lightweight to ensure efficient training while maintaining sufficient expressive power.

Key hyperparameters include the embedding dimension, number of attention heads, number of Transformer layers, and feed-forward hidden dimension. In the base configuration, the embedding dimension is set to 128, with 4 attention heads and 2 Transformer layers. The feed-forward network expands the hidden dimension to 256 before projecting it back to 128. The vocabulary size is fixed at 500 tokens, and the context length is set to 50. These parameters are varied in later experiments to study their impact on model performance.

The output of the final layer is projected onto the vocabulary space using a linear layer, producing a probability distribution over possible next tokens for each position in the sequence.

The overall architecture of the model is illustrated in Figure 1.

E. Debugging Against a Reference Implementation

To verify the correctness of the custom self-attention implementation, its outputs are compared with PyTorch's `nn.MultiheadAttention` under identical inputs and aligned projection weights.

Both implementations produce outputs of shape `[2, 6, 128]`. The maximum absolute difference is 1.78×10^{-7} for outputs and 5.96×10^{-8} for attention weights, indicating near-identical behavior.

This comparison confirms the correctness of the custom attention module and is used solely for debugging purposes.

IV. Training Setup and Evaluation

A. Training Objective

The model is trained using the next-token prediction objective, where the goal is to maximize the likelihood of the correct next token given the preceding context. This task is formulated

as a multi-class classification problem at each position in the sequence.

Cross-entropy loss is used as the optimization objective, measuring the discrepancy between the predicted probability distribution over the vocabulary and the true target tokens. Minimizing this loss encourages the model to assign higher probabilities to correct token predictions and learn meaningful representations of the underlying language structure.

B. Training Setup

To ensure reproducibility and clarity, the base training configuration is summarized below.

Optimizer: Adam optimizer is used due to its adaptive learning rate properties and stable convergence behavior.

Learning Rate: The base learning rate is set to 3×10^{-4}, which provides a good balance between convergence speed and stability.

Batch Size: A batch size of 64 is used across all experiments.

Number of Epochs: The model is trained for 10 epochs, with training and validation metrics recorded at each epoch.

Gradient Clipping: Gradients are clipped with a maximum norm of 1.0 to prevent exploding gradients and improve training stability.

Best Epoch: The best-performing model is selected based on the lowest validation loss, which occurs at the final epoch in the base configuration.

Device: Training is performed on a GPU when available, otherwise falling back to CPU.

Reproducibility: To ensure consistent and reproducible results, a fixed random seed of 42 is used across all experiments. This seed is applied to model initialization, data shuffling, and other stochastic operations, reducing variability between runs.

This configuration serves as the baseline for all experiments, with individual hyperparameters (e.g., learning rate, context length, and model size) varied independently to study their effects on model performance.

C. Optimization Details

Model parameters are optimized using the Adam optimizer with mini-batch gradient descent. At each iteration, gradients of the cross-entropy loss are computed via backpropagation and used to update model parameters. Gradient clipping with a maximum norm of 1.0 is applied to mitigate potential exploding gradients and ensure stable training.

Validation loss is monitored throughout training to assess generalization performance and detect potential overfitting. The final model is selected based on the lowest validation loss achieved during training.

No explicit regularization techniques such as dropout or weight decay are applied, as the relatively small model size and dataset provide sufficient implicit regularization.

D. Evaluation Metric: Perplexity

Model performance is evaluated using perplexity, a standard metric for language modeling tasks. Perplexity is defined as the exponential of the average cross-entropy loss:

$$\text{Perplexity} = \exp(\mathcal{L}_{\text{val}})$$

where $\mathcal{L}_{\text{val}}$ denotes the validation loss. Perplexity measures how well the model predicts unseen data, with lower values indicating better performance.

Intuitively, perplexity reflects the average uncertainty of the model when predicting the next token. A lower perplexity indicates that the model assigns higher probability to the correct tokens and has learned more accurate representations of the underlying language structure.

V. Experimental Results

A. Training Dynamics

The training and validation loss curves over epochs are shown in Figure 2. Both curves exhibit a consistent downward trend, indicating that the model successfully learns meaningful patterns from the data. The gap between training and validation loss remains moderate, suggesting that overfitting is limited and generalization is maintained.

At later stages of training, the validation loss stabilizes, indicating convergence. This behavior suggests that the chosen training configuration provides a good balance between learning capacity and regularization.

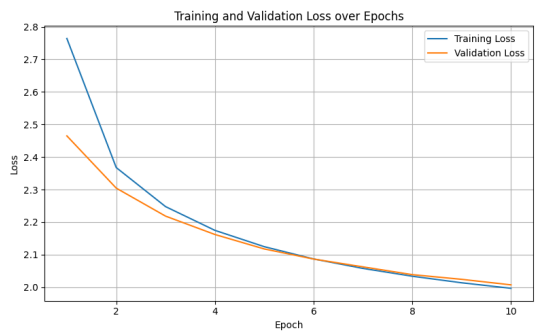

Fig. 2: Training and validation loss over epochs.

B. Validation Performance

The best validation performance achieved corresponds to a validation loss of 2.0073, resulting in a perplexity of 7.4431. These values indicate that the model is able to assign relatively high probability to the correct next tokens despite its compact size.

The achieved perplexity reflects the model's ability to capture both local and short-range dependencies in the text. However, due to the limited model size and context length, performance on longer-range dependencies remains constrained.

C. Effect of Hyperparameters

The impact of key hyperparameters on model performance is analyzed, including learning rate, context length, and model size, as illustrated in Figure 3. The experimental settings used for this analysis are summarized in Table I.

The learning rate is observed to have the most significant effect on training stability and convergence. Smaller learning

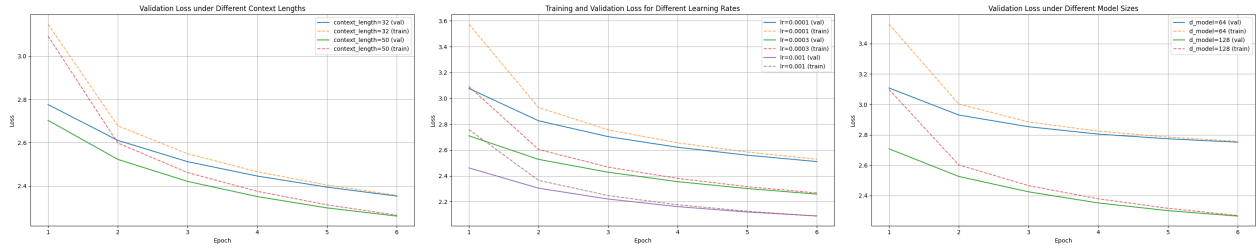

Fig. 3: Effect of key hyperparameters on model performance. Left: Validation loss under different context lengths. Middle: Training and validation loss for different learning rates. Right: Validation loss under different model sizes.

rates result in slower but more stable optimization, while larger values accelerate learning but may introduce instability. An intermediate learning rate provides the best balance between convergence speed and stability.

Increasing the context length improves performance by enabling the model to capture longer-range dependencies, leading to lower validation loss. However, this comes at a higher computational cost due to the quadratic complexity of the attention mechanism.

Similarly, larger model sizes improve performance by increasing representational capacity, resulting in lower validation loss. These gains, however, require increased memory and training time.

Overall, the learning rate is identified as the most sensitive hyperparameter, while context length and model size primarily enhance performance through increased modeling capacity, highlighting a trade-off between efficiency and accuracy.

TABLE I: Hyperparameter configurations explored in experiments.

Hyperparameter	**Values**
Learning rate	$1e{-}4,\ 3e{-}4,\ 1e{-}3$
Context length	$32,\ 50$
Model size (d_{model})	$64,\ 128$

D. Qualitative Assessment with Sample Generation

In addition to quantitative results, qualitative assessment is performed through sample generation to evaluate model fluency and coherence. While validation perplexity remains the primary evaluation metric, generated text provides insight into whether the model has learned locally plausible Shakespeare-like continuations.

To assess this behavior, text is generated using short, medium, and long prompts with a maximum generation length of 60 tokens.

For short prompts such as *"To be"*, the model produces fluent continuations with a structure resembling Shakespearean dialogue:

> *To be my hand, Is time he cans and endlse to show the strength of sixother, And yet, as he had quite rocky youth seven. If it were more caught, and*

For medium prompts such as *"The king is wise and"*, the model maintains grammatical coherence and continues the sentence in a contextually plausible manner, while also introducing stylistic elements such as speaker annotations:

> *The king is wise and still that speaks the fair. KING RICHARD II: Faith; for I will discredd; for, those bloody look'd For I have found'd young to-night? Here*

For longer prompts such as *"To be or not to be, that is the question"*, stylistic consistency is preserved, but coherence degrades slightly over longer sequences:

> *To be or not to be, that is the question of that appear ox, To aged my brother's face and fees Will serve first beat proves not And credulously loving brother Clarence. MENENIUS*

Overall, the model demonstrates strong stylistic learning and reasonable fluency for shorter contexts, while coherence decreases for longer sequences due to occasional nonsensical phrases and semantic drift. This behavior is expected given the limited model size and training data.

VI. Attention Analysis and Interpretation

A. Attention Patterns in Sample Sequences

Attention heatmaps are visualized to examine how the model distributes focus across tokens when making predictions. Representative examples are shown in Figure 4. Each row of the heatmap corresponds to a query token, while each column represents the tokens being attended to.

A clear lower-triangular structure is observed, reflecting the effect of causal masking, where each token attends only to itself and preceding tokens. Within this constraint, attention weights are not uniformly distributed. Instead, stronger weights are assigned to nearby tokens and syntactically relevant words, indicating that the model learns to prioritize meaningful context for prediction.

These patterns suggest that the model effectively captures local dependencies and short-range contextual relationships, which are essential for coherent next-token prediction.

B. Evolution of Attention During Training

To better understand how attention develops over time, attention maps are visualized at different training epochs using the sentence *"The man who spoke to the king is here"*, as shown in Figure 5. This example introduces non-trivial syntactic structure, including a relative clause and long-range

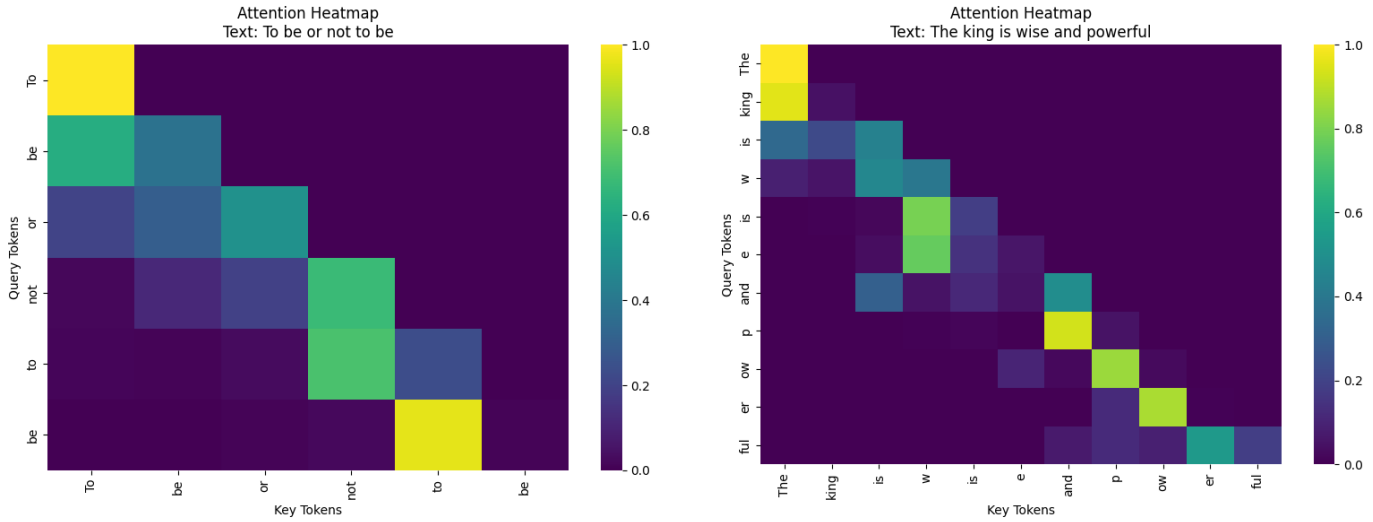

Fig. 4: Attention heatmaps for sample sequences. Left: *"To be or not to be"*. Right: *"The king is wise and powerful"*.

dependencies, enabling a more detailed analysis of attention behavior.

In early training stages (epoch 1), attention distributions appear diffuse and broadly distributed across available tokens. The model has not yet learned meaningful relationships, and attention is spread over multiple positions without clear preference.

As training progresses (epoch 5), attention becomes increasingly structured. Distinct patterns begin to emerge, with tokens focusing more selectively on specific positions. In particular, tokens within the relative clause ("who spoke to the king") start to exhibit stronger internal attention, indicating that the model is beginning to capture local syntactic structure.

By later epochs (epoch 10), attention maps exhibit sharp and highly concentrated patterns. Tokens consistently attend to a small number of relevant positions, reflecting increased confidence and reduced uncertainty. Notably, long-range dependencies emerge, such as attention from the verb "is" back to the subject "man", suggesting that the model has learned aspects of subject–verb relationships. Additionally, attention within the relative clause becomes more coherent, indicating improved representation of hierarchical structure.

Overall, these results demonstrate a clear transition from unstructured attention to organized and linguistically meaningful patterns, highlighting how the model progressively learns to identify and exploit relevant contextual information during training.

C. Layer and Head Behavior

Different attention layers and heads exhibit distinct and complementary patterns of behavior. Lower layers (e.g., Layer 1) primarily focus on local dependencies, with attention concentrated around neighboring tokens and short phrases. This behavior reflects the model's early-stage processing of local contextual information.

In contrast, deeper layers (e.g., Layer 2) display more selective and globally distributed attention patterns. Tokens in these layers attend more strongly to semantically or syntactically relevant positions, even when they are not adjacent. For example, attention from later tokens such as "is" often links back to earlier tokens such as "man", indicating that deeper layers capture longer-range dependencies within the sequence.

Furthermore, different attention heads within the same layer exhibit clear specialization. Some heads emphasize positional or near-diagonal patterns, effectively modeling local continuity, while others focus on specific tokens or syntactic structures, such as interactions within the relative clause ("who spoke to the king"). This diversity allows the model to represent multiple types of relationships simultaneously and aggregate them into richer contextual representations.

Overall, these observations highlight that attention mechanisms not only enforce causal structure but also enable the model to dynamically organize information across different scales, from local token interactions to broader sentence-level dependencies. This layered and multi-head specialization plays a critical role in the model's ability to learn meaningful representations and improve predictive performance.

VII. Discussion and Reflection

A. Learned Patterns and Model Behavior

The trained Transformer model demonstrates the ability to capture meaningful linguistic patterns from the dataset. Attention visualizations indicate that the model effectively focuses on relevant prior tokens, particularly those that are syntactically or semantically related to the current prediction. This behavior enables the model to generate coherent local structures and maintain stylistic consistency characteristic of the Shakespeare corpus.

However, the model primarily captures short-range dependencies, with limited ability to model long-range relationships. This limitation is largely attributed to the relatively small

context length and compact architecture, which restrict the amount of information available for each prediction.

B. Runtime and Memory Considerations

The computational cost of the model is primarily dominated by the self-attention mechanism, whose attention score tensor has shape $[B, H, T, T]$ and therefore scales quadratically with sequence length. This behavior is reflected in both memory usage and runtime characteristics, summarized in Table II.

TABLE II: Runtime and memory comparison across configurations.

Setting	Time (s)	Memory (MB)
Ctx = 32	65.52	8.00
Ctx = 50	67.20	13.38
$d_{model} = 64$	67.46	7.91
$d_{model} = 128$	67.29	13.38

As shown in Table II, increasing the context length leads to a substantial increase in memory usage, rising from 8.00 MB to 13.38 MB when moving from length 32 to 50. In contrast, the runtime per epoch changes only slightly (65.52s to 67.20s), indicating that sequence length has a stronger impact on memory than on runtime.

Similarly, increasing the model size from $d_{model} = 64$ to $d_{model} = 128$ results in a significant increase in memory usage (7.91 MB to 13.38 MB), while the runtime remains nearly unchanged (67.46s vs. 67.29s).

These results indicate that memory consumption is substantially more sensitive to both sequence length and model size, whereas runtime remains relatively stable in this setting. Overall, the primary computational bottleneck arises from the quadratic scaling of attention with respect to sequence length, while model size contributes mainly to increased memory usage rather than runtime.

C. Limitations

Several limitations are identified in this work. The model size is relatively small, limiting its ability to capture complex and long-range dependencies. The dataset is also limited in scale, which restricts the diversity of learned language patterns. Additionally, while attention visualizations provide useful insights, they do not fully explain all aspects of model behavior.

Despite these limitations, the model successfully demonstrates the core principles of Transformer-based language modeling in a computationally efficient setting.

VIII. Conclusion

In this work, a lightweight Transformer-based language model is implemented from scratch and applied to the task of next-token prediction on the Tiny Shakespeare dataset. The model successfully learns meaningful patterns in the data, as evidenced by decreasing training and validation loss, as well as competitive perplexity values.

The results demonstrate that even a compact Transformer architecture is capable of capturing important linguistic structures, particularly local dependencies and stylistic patterns. Analysis of attention mechanisms reveals that the model learns to focus on relevant contextual tokens and develops increasingly structured attention patterns over the course of training.

The impact of key hyperparameters, including learning rate, context length, and model size, highlights important trade-offs between performance, stability, and computational efficiency. Additionally, the role of positional encoding is shown to be essential for preserving sequence order and enabling effective language modeling.

While the model is limited by its small size and restricted context window, it provides a clear and practical demonstration of the core principles underlying modern Transformer-based language models. Overall, this work offers valuable insight into how attention-based architectures learn and operate in natural language tasks.

AI Tool Usage Disclosure

AI Tool Used: ChatGPT (OpenAI)

ChatGPT was used as a supportive development tool during this project. Specifically, it assisted with converting written content into IEEE LaTeX format, restructuring experimental pipeline code to improve clarity and reproducibility, verifying Python syntax and library usage, and refining paragraphs for grammatical correctness and linguistic clarity.

All experimental design decisions, feature engineering choices, model selection strategies, evaluation procedures, result interpretation, and analysis were independently conceived, implemented, and validated by the author. The AI tool was used solely as an auxiliary formatting, coding, and language improvement assistant, and did not determine methodological or analytical conclusions.

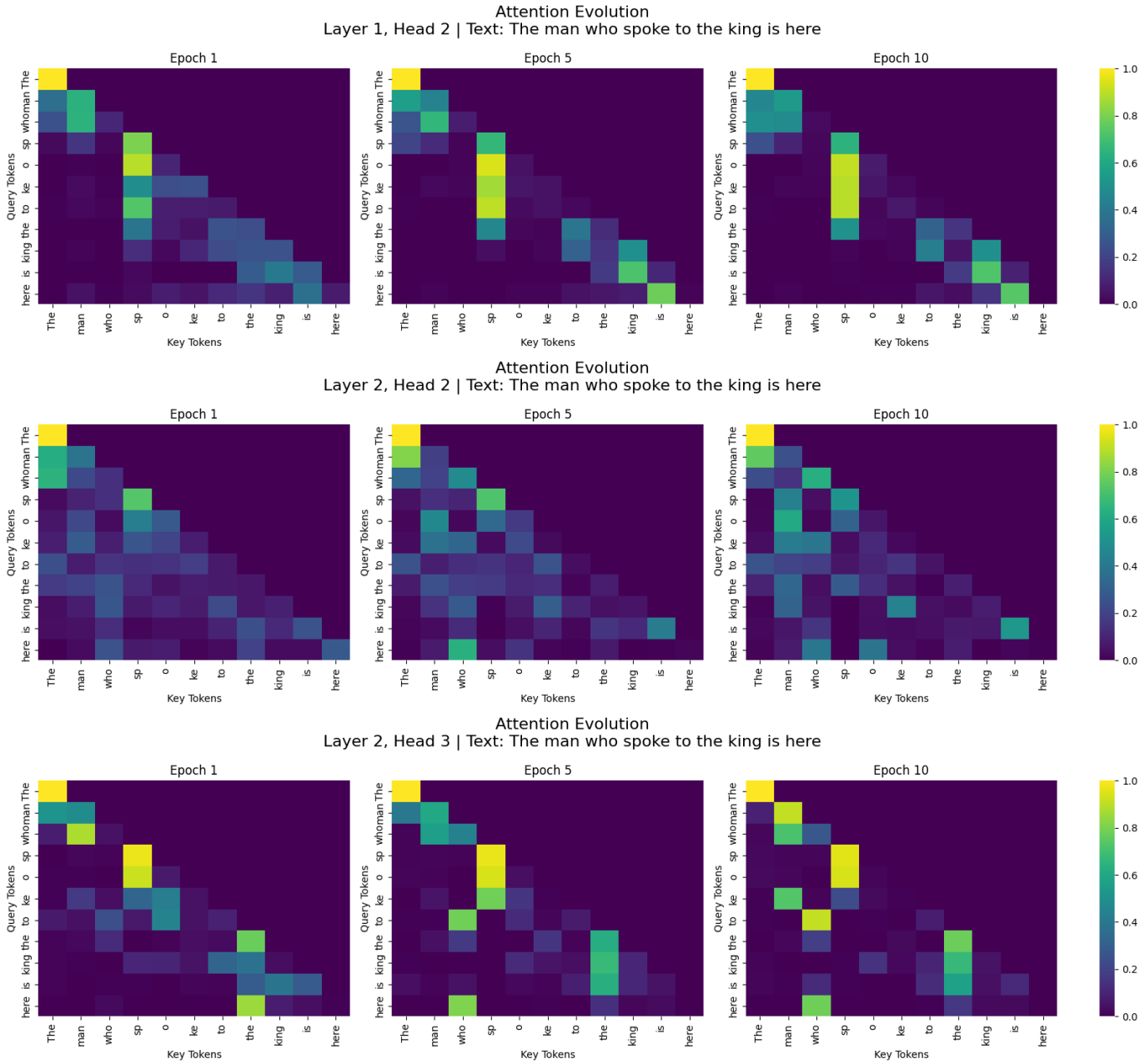

Fig. 5: Evolution of attention patterns across training epochs for different layers and heads using the sentence "The man who spoke to the king is here". Top: Layer 1, Head 2. Middle: Layer 2, Head 2. Bottom: Layer 2, Head 3. Each row shows attention maps at early (epoch 1), intermediate (epoch 5), and later (epoch 10) stages of training. Attention transitions from diffuse distributions to concentrated and structured patterns. Lower layers primarily capture local dependencies, while deeper layers exhibit more selective and longer-range attention, including relationships between distant tokens such as the subject ("man") and verb ("is"). Differences across heads highlight specialization, with some focusing on local positional structure and others capturing broader syntactic relationships.